Programming Exercise 1: Hypothesis

Machine Learning

Supervised Algorithms

UNIVERSIDAD DE SAN BUENAVENTURA

Guide #1- Numpy y Matplotlib

Introduction

The aim of this guide is to understand how Numpy and matplotlib works.

To get started with this exercise, there are a series of steps with different examples of how to use both libraries and its instructions.

1. Use of Numpy library

Numpy is a fundamental Python library for topics related to numerical computing, data science, optimization engineering, among others. It uses a multidimensional array data structure, in which different functions and tools for working with arrays are integrated.

To continue with the proposed exercises, you can go to the following link:
https://numpy.org/doc/stable/user/quickstart.html

At first, it is important to install the library in the computer by writing the following instruction in the console log:

```
pip install numpy
```

2. Numpy instructions

Numpy's array class is called *ndarray*. To use this library is indispensable to import the library on the top of your program, as follows:

```
import numpy as np
```

The instruction *np.arange* allows to create a dimensional matrix ordered by the instruction reshape(n ,m), where n is the number of rows and m is the number of columns.

```
a = np.arange(15).reshape(3, 5)
        >>([[ 0,  1,  2,  3,  4],
            [ 5,  6,  7,  8,  9],
            [10, 11, 12, 13, 14]])
b =  np.arange(10, 30, 5)
        >>([10, 15, 20, 25])
```

With the instruction *shape* it is possible to know the size of the matrix, as follow:

```
Print(a.shape)
      >>(3, 5)
```

Also, the instruction *ndim* gives valuable information about the dimensions of the matrix and *type* indicates the variable type of a variable.

```
Print(a.ndim)
      >>2
Print(type(a))
      >><class 'numpy.ndarray'>
```

Besides, the instruction *array* allows to create a specific matrix, by specifying the internal values.

```
b = np.array([6, 7, 8])
print(b)
      >>([6, 7, 8])
```

There are 3 different functions used to create arrays full of zeros, full of ones or filled with random numbers. *Zeros*, *ones* and *empty* respectively.

```
np.zeros((3, 4))
      >>([[0., 0., 0., 0.],
          [0., 0., 0., 0.],
          [0., 0., 0., 0.]])

np.ones((2, 3, 4), dtype=np.int16)
      >>([[[1, 1, 1, 1],
          [1, 1, 1, 1],
          [1, 1, 1, 1]],
         [[1, 1, 1, 1],
          [1, 1, 1, 1],
          [1, 1, 1, 1]]], dtype=int16)

np.empty((2, 3)) #may vary
       >>([[3.73603959e-262, 6.02658058e-154, 6.55490914e-260],
           [5.30498948e-313, 3.14673309e-307, 1.00000000e+000]])
```

3. Use of Matplotlib library

Matplotlib is a library used to create static, animated and interactive visualizations in Python. Widely used for generating plots, charts, histograms and more to present data. Its versatility is perfect to represent data in engineering.

To continue with the proposed exercises, you can go to the following link: https://matplotlib.org/stable/users/getting_started/

At first, it is important to install the library in the computer by writing the following instruction in the console log:

```
pip install matplotlib
```

UNIVERSIDAD DE SAN BUENAVENTURA

4. Matplotlib instructions

Generally, in this course will be widely used the module of *pyplot* to create graphics, like MATLAB.

```
import matplotlib.pyplot as plt
```

The function *subplots* creates a graphic by using different coordinates, such as x-y, polar plot or 3D. Note that at the end it is necessary to write *plt.show( )*, in order to display the data.

```
fig, ax = plt.subplots(figsize=(5,5))  # Create a figure
containing a single axes.
ax.plot([1, 2, 3, 4], [1, 4, 2, 3])  # Plot some data on the
axes.
#---------------- To display the graph -------------------
plt.show() #write it at the end of this code
```

Fig and ax are childs of the figure, that can be used to specify more than one graphic in the same figure.

```
fig, axs = plt.subplots(2, 2, figsize=(9, 5)) # Create a figure
containing 4 subplots, (2 rows, 2 columns)

ax_1 = fig.add_subplot(1,2, 1) # Combine to position in the
graph, add_subplot(row_pos,col_pos,1st_idx)
ax_1.plot([1, 2, 3, 4], [1, 4, 2, 3], label='Plot 1 & 3')
ax_1.plot([1, 2, 3, 4], [1, 2, 3, 4])
axs[0, 1].plot([1, 2, 3, 4], [1, 2, 1, 2])
axs[1, 1].plot([1, 2, 3, 4], [4, 3, 2, 1])
```

To label the axes of a figure, there are the following instructions added to the previous code.

```
ax_1.set_title('Combined Plot 1 & 3')#Set the title for ax_1
ax_1.set_xlabel('Time')  #Set the xlabel for ax_1
ax_1.set_ylabel('Amplitude')  #Set the ylabel for ax_1
axs[0, 1].set_title('Plot 2')
axs[0, 1].set_xlabel('Time')
axs[0, 1].set_ylabel('Amplitude')
axs[1, 1].set_title('Plot 4')
axs[1, 1].set_xlabel('Time')
axs[1, 1].set_ylabel('Amplitude')
axs[0, 0].remove()   #Remove the plot[0,0]
axs[1, 0].remove()    #Remove the plot[1,0]
plt.tight_layout()   #adjust the space within graphs
```

The function *scatter* helps to represent the data with specific symbols; besides it is possible to include a legend that helps to understand the data shown.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()  # a figure with a single Axes
```

```
x=[1,5,4,8,3]; y=[3,4,5,6,2]
ax.scatter(x, y, marker='x', c='r', label="values")
ax.legend(loc='upper rigth')
plt.show()
```

5. Array indexation.

Array Indexation refers to the process of accessing the different elements or subsets that are within an array. In Python, it is possible to manipulate this data by using square brackets *[ ]* after the specified array.

a) **Accessing To elements:** To access each element in the array, is typically used square brackets, by pointing its row and column indices. Ex:

```
import numpy as np
a=np.arange(15).reshape(5,4)
print('Array: ')
print(a)
print(f'\n(3rd row, 2nd col): {a[3,2]}')
```

b) **Slicing rows and columns:** it allows to extract an entire row or column from the array by using the colon operator ':'. Ex:

```
print(a[:,:2],'\n')    #[all rows, columns 0 to 1]
print(a[:,2:],'\n')    #[all rows, columns 2 to the end]
print(a[2:4,:],'\n')   #[rows 2 and 3, all columns]
print(a[0,:],'\n')     #[row 0, all columns]
print(a[1:,:],'\n')    # all data except the first row
print(a[:-3])          # first 2 rows of the array
print(a[-3:])          # last 3 rows of the array
print(a[::2, :])       # [Select every second row, all
columns]
```

6. Array instructions.

To operate matrixes, there are different kinds of operators such as product, transpose, and so on.

a) Transpose:
```
import numpy as np
a=np.arange(20).reshape(5,4)
print(a.T)
```

b) Product:
```
b=np.arange(20).reshape(4,5)
print(a@b)
```

c) Scalar multiplication:
```
print(2*b)
```

d) Matrix Boolean identification:
```
print(a<9)
```

e) Nonzero elements:
```
c=np.nonzero(a%9==0)
print(c)
```

f) Identity matrix
```
print(np.eye(3))
```

7. Reading files.

In Python it is possible to open plain files, such as .txt, .xml, so on. These files contain binary data that can be extracted in Python to organize or manipulate elements. The instruction to extract information is as follows:

```
data = np.loadtxt(filename, delimiter=',',dtype=float)
```

delimiter: indicates the element within the data that separates them individually in the dataset.
dtype: indicates the type of data that Python should read.

Note: to open any file it is needed that the dataset file should be in the same folder of the project, on other case, it is possible to specify the route by using “ and //. Ex:

```
“Practice_Lab_ex1\\data\\exdata.txt"
```

Programming Exercise 1: Hypothesis

Machine Learning

Supervised Algorithms

UNIVERSIDAD DE SAN BUENAVENTURA

Practice #1

Files included.

Lab1.py – *main script to execute the practice #1*
[*] Plotting.py – *Function to plot the dataset.*
[*] Reading.py – *Function to read the plain file of data.*
[*] Store_Sales.py – *Function to manipulate the information.*

* Indicates files you will need to complete.

1. Data reading function.

The first part of the Lab is to code the function ***dataReading()*** in charge of reading the dataset in folder "*dataset*". The function should return the whole dataset for the files Inventory.csv & Sales.csv.

Expected output:

```
These are the first 3 rows of the dataset __Sales__:

[['SalesId' 'StoreId' 'ProductId' 'Date' 'UnitPrice'
'Quantity']
 ['82319' '22726' '590' '2019-12-02' '0.0525' '93']
 ['15022' '21754' '390' '2017-11-19' '5.109999999999999'
'28']]

These are the first 3 rows of the dataset __Inventory__:

[['ProductId' 'StoreId' 'StoreName' 'Address' 'neighborhood'
  'QuantityAvailable']
 ['1' '85123' 'National Stores' '9 Springview Point' 'Bolton
Hill' '11']
 ['2' '71053' 'Family Dollar' '5434 Daystar Circle'
'Ashburton' '1']]
```

2. Data manipulation.

After completing the first part, the data is available to be manipulated, to create statistics, studies or plotting diagrams.

In this part the task is to code a function ***stores()*** to get the global sale values for the following columns (*SalesId, UnitPrice, Quantity*). Return a list of float elements with just those three columns.

Expected output:

```
Dataset size: (200000, 3)
[[8.2319e+04 5.2500e-02 9.3000e+01]
```

```
[1.5022e+04 5.1100e+00 2.8000e+01]
[1.1624e+04 7.3675e+00 3.3000e+01]
[6.3101e+04 2.0825e+00 7.6000e+01]
[2.9702e+04 6.4750e-01 8.0000e+00]
[3.5660e+04 5.6700e+00 9.3000e+01]
[6.9913e+04 4.3750e-01 9.8000e+01]
[4.7278e+04 9.2050e+00 1.7000e+01]
[4.6126e+04 3.3950e+00 4.7000e+01]
[6.9718e+04 1.5925e+00 4.3000e+01]]
```

3. Data specification.

After finishing the previous step, order the data set in two different arrays, to plot it later in individual graphs.

Complete the code in the function ***info()*** to return individual dataset *[StoreId, UnitPrice]* & *[StoreId,Quantity]*

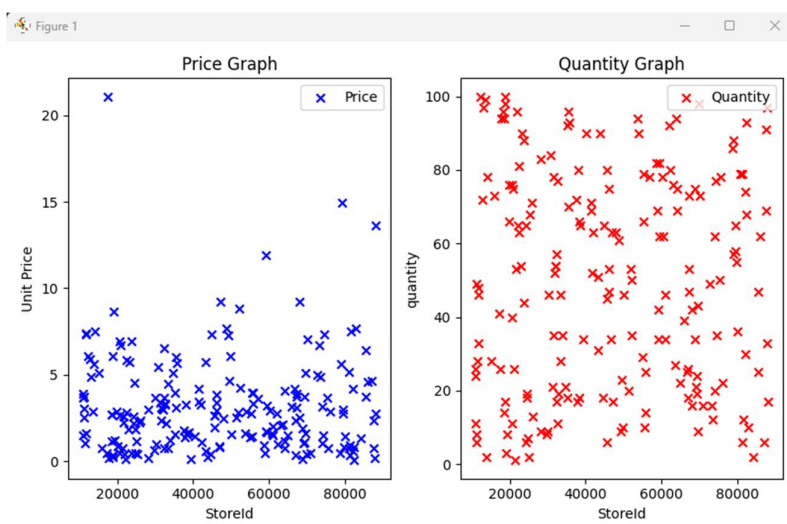
Expected output:

```
Unit Price dataset ...
[[8.2319e+04 5.2500e-02]
 [1.5022e+04 5.1100e+00]
 [1.1624e+04 7.3675e+00]
 [6.3101e+04 2.0825e+00]
 [2.9702e+04 6.4750e-01]]
```

```
Quantity dataset ...
[[8.2319e+04 9.3000e+01]
 [1.5022e+04 2.8000e+01]
 [1.1624e+04 3.3000e+01]
 [6.3101e+04 7.6000e+01]
 [2.9702e+04 8.0000e+00]]
```

4. Plotting data.

Finally plot the data in the same figure but in different subplots for the price and the quantity by completing the function ***dataview()***.



Finally, submit your work in the virtual platform.